

Q5.3 Given $Q_\theta(s, a) = \theta^T \phi(s, a) = \sum_{k=1}^2 \theta_k \phi_k(s, a) \quad \text{--- ①}$

with feature $\phi = [\phi_1(s, a) \quad \phi_2(s, a)]^T$

$$\theta = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} \in \mathbb{R}^2$$

$$\phi_1(s, a) = -2 \mathbb{1}[s=b] - \mathbb{1}[s=c] - 0.5 \mathbb{1}[s=d]$$

$$\phi_2(s, a) = \mathbb{1}[a=x] - \mathbb{1}[a=y]$$

part 5.2 has dataset $\mathcal{D} = \{(s_t, a_t, y_t)\}_{t=1}^3 \quad N=3$

input(s, a)	y_t target
c, x	1.675
e, x	-3.35
b, y	-1.85

from part 5.2

we know $y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a') \quad \text{--- ②}$

also empirical risk minimization objective

$$L(\theta_1, \theta_2) = \sum_{t=1}^3 (\theta_1 \phi_1(s_t, a_t) + \theta_2 \phi_2(s_t, a_t) - y_t)^2 \quad \text{--- ③}$$

Since from ① we substitute $Q_\theta(s, a) = \theta_1 \phi_1(s, a) + \theta_2 \phi_2(s, a) \quad \text{--- ④}$

Computing features for each datapoint

① $(s, a) = (c, x) \quad \left| \quad \begin{aligned} \phi_2(c, x) &= 1 - 0 = 1 \\ \phi_1(c, x) &= -1 \end{aligned} \right.$

② $(s, a) = (e, x) \quad \left| \quad \begin{aligned} \phi_2(e, x) &= 1 \\ \phi_1(e, x) &= 0 \end{aligned} \right.$

③ $(s, a) = (b, y) \quad \left| \quad \begin{aligned} \phi_2(b, y) &= -1 \\ \phi_1(b, y) &= -2 \end{aligned} \right.$

eqn ④ now updates as,

$$Q_\theta(c, x) = \theta_1(-1) + \theta_2(1) = -\theta_1 + \theta_2$$

$$Q_\theta(e, x) = \theta_1(0) + \theta_2(1) = \theta_2$$

$$Q_\theta(b, y) = \theta_1(-2) + \theta_2(-1) = -2\theta_1 - \theta_2$$

plug expressions in (3)

$$L(\theta_1, \theta_2) = \frac{1}{3} \left[(\theta_1(c, x) + y_1)^2 + (Q_\theta(e, x) - y_2)^2 + (Q_\theta(b, y) - y_3)^2 \right] \quad (2)$$

$$= \frac{1}{3} \left[(-\theta_1 + \theta_2 - y_1)^2 + (\theta_2 - y_2)^2 + (-2\theta_1 - \theta_2 - y_3)^2 \right]$$

$$L(\theta_1, \theta_2) = \frac{1}{3} \left[(-\theta_1 + \theta_2 - 1.675)^2 + (\theta_2 + 3.35)^2 + (-2\theta_1 - \theta_2 + 1.85)^2 \right]$$

N.B. mean empirical square loss

$$L(\theta_1, \theta_2)_{MSL} = \frac{1}{3} \left[(-\theta_1 + \theta_2 - 1.675)^2 + (\theta_2 + 3.35)^2 + (-2\theta_1 - \theta_2 + 1.85)^2 \right] \quad \leftarrow [Answer]$$

P2.1

To show tabular representation is a special case of linear class func $Q_\theta(s, a) = \theta^T \phi(s, a)$ we will define the finite MDP with state space S & action space A with set of features $\phi(s, a)$ & parameters θ that replicate the tabular $Q(s, a)$ values. Let total number of state-action pairs be $k = |S| \times |A|$ we create one to one mapping of indices from each (s, a) pair to integers $i \in \{1, 2, \dots, k\}$ so that this vector k feature vectors will be indicator vectors or one-hot encoding which gives one feature per pair.

Next, we can index each pair $(s_1, a_1), (s_1, a_2), \dots, (s_{|S|}, a_{|A|})$ where $l(s, a) \rightarrow$ index be index pair (s, a) . we next define $\theta \in \mathbb{R}^k$

where $\theta = \begin{bmatrix} Q(s_1, a_1) \\ \vdots \\ Q(s_{|S|}, a_{|A|}) \end{bmatrix}$ so that $\theta_{l(s, a)}$ stores the tabular value for pair (s, a)

For each (s, a) , $\phi(s, a) \in \mathbb{R}^k$ one-hot indicator vector.

$$\phi_i(s, a) = \begin{cases} 1 & \text{if } i = l(s, a) \\ 0 & \text{otherwise} \end{cases}$$

Linear approximation

$$Q_\theta(s, a) = \theta^T \phi(s, a) = \sum_{i=1}^k \theta_i \phi_i(s, a)$$

Since $\phi(s, a)$ is one-hot $Q_\theta(s, a) = \theta_{l(s, a)}$ [other terms zero]

$$\text{since } \theta_{l(s, a)} = Q(s, a)$$

$$\text{so } Q_\theta(s, a) = Q(s, a) \quad \forall (s, a)$$

$\therefore$ linear π class reproduces tabular Q -fn

e.g. $S = \{s_1, s_2\}$ $A = \{a_1, a_2\}$. $k = 4$ state-action pairs indexed as (3)

1. (s_1, a_1)
2. (s_1, a_2)
3. (s_2, a_1)
4. (s_2, a_2)

$$\Delta \theta = \begin{bmatrix} Q(s_1, a_1) \\ Q(s_1, a_2) \\ Q(s_2, a_1) \\ Q(s_2, a_2) \end{bmatrix}, \quad \phi(s_1, a_1) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\therefore Q_\theta(s_1, a_1) = [Q(s_1, a_1) \dots Q(s_2, a_2)] \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} = Q(s_1, a_1)$$

$$\therefore Q_\theta(s_1, a_1) = Q(s_1, a_1) \quad \text{---x---} \quad \text{Q.E.D. \& demonstrated.}$$

P2.2 Given, $S = S_1 \cup S_2 \cup \dots \cup S_n$ s.t. $S_{j_1} \cap S_{j_2} = \emptyset$ ($j_1 \neq j_2$)
 $A = A_1 \cup A_2 \cup \dots \cup A_m$, $A_{j_1} \cap A_{j_2} = \emptyset$ ($j_1 \neq j_2$)
 S_i groups have similar states & A_j groups have similar action. We are asked for a class of linear functions
 $Q = \{Q_\theta : Q_\theta(s, a) = \theta^T \phi(s, a) \forall s \in S \& a \in A, \theta \in \mathbb{R}^k$

We can define feature map $\phi(s, a)$ for each pair (i, j) as before,
 $\phi_{ij}(s, a) = \begin{cases} 1 & \text{if } s \in S_i \& a \in A_j \\ 0 & \text{otherwise.} \end{cases}$

each pair group (S_i, A_j) get one feature & one parameter
 $\therefore$ there are $n \cdot m$ possible pairs $\therefore$ total feature $k = nm$

Stacking all $\phi_{ij}(s, a)$ we get $\phi(s, a) \in \mathbb{R}^{nm}$ which is one-hot vector because each (s, a) belong to exactly one pair of groups.

let $\theta \in \mathbb{R}^{nm}$ where θ_{ij} entries correspond to each group pair (S_i, A_j) $\therefore Q_\theta(s, a) = \theta^T \phi(s, a) = \theta_{ij}$ when $s \in S_i$ & $a \in A_j$

$\therefore$ all state action pairs inside the same group pair share the same Q value

To show with an example $S_1 = \{s_1, s_2\}$ $A_1 = \{a_1\}$
 $S_2 = \{s_3, s_4\}$ $n=2$ grp

$$A_1 = \{a_1\}, A_2 = \{a_2, a_3\} \quad m=2 \text{ groups}$$

(4)

$$k = nm = 2 \times 2 = 4$$

$$\Theta = \begin{bmatrix} \theta_{11} \\ \theta_{21} \\ \theta_{12} \\ \theta_{22} \end{bmatrix} \quad \theta_{ij} \rightarrow s_i \times A_j \text{ for each}$$

for (s_a, a_1) , $s_a \in S_1, a_1 \in A_1$ - partition block $\phi = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$ feature vector $\therefore Q_\Theta(s, a) = \Theta^T \phi(s, a) = \theta_{11}$

for $(s_b, a_1), \dots, (s_1, a_1)$, $\phi = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$ $Q_\Theta = \theta_{11}$ } same

for $(s_c, a_3), \dots, (s_2, a_2)$, $\phi = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$ $Q_\Theta = \theta_{12}$ } same

for $(s_d, a_3), \dots, (s_2, a_2)$, $\phi = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$ $Q_\Theta = \theta_{22}$ } same

$\therefore S \times A$ block of state & action e.g. (s_a, a_1) and (s_b, a_1) & $\phi(s, a)$ is identical forces linear form $Q_\Theta(s, a)$ to output same value θ_{11}

Q.E.D.

P2.3 Dataset $\{x^i = (s^i, a^i), y^i\}_{i=1}^N$, y^i sample returns. for also $Q_\Theta(s, a) = \Theta^T \phi(s, a)$, $\phi(s, a) \in \mathbb{R}^k$ & $\phi(s, a): S \times A \rightarrow \mathbb{R}^k$

$$\text{ask is } \hat{Q}^\pi(s, a) = \arg \min_{Q \in \mathcal{Q}} \sum_{i=1}^N (Q(s^i, a^i) - y^i)^2$$

also $\sum_{i=1}^N \phi(s^i, a^i) \phi^T(s^i, a^i)$ forms invertible matrix.

$$\text{so the objective function } J(\Theta) = \sum_{i=1}^N (\Theta^T \phi(s^i, a^i) - y^i)^2 \quad \text{--- ①}$$

this is quadratic & convex so $\nabla_\Theta J(\Theta) = 0$ is global minimizer.

First convert Θ to matrix form.

$$\Phi = \begin{bmatrix} \phi(s^1, a^1)^T \\ \phi(s^2, a^2)^T \\ \vdots \\ \phi(s^N, a^N)^T \end{bmatrix}$$

$$y = \begin{bmatrix} y^1 \\ \vdots \\ y^N \end{bmatrix} \quad N \times 1 \text{ matrix}$$

$\Theta \Rightarrow k \times 1$ col vector to solve

loss f ①

5

$$J(\theta) = \|\Phi\theta - y\|_2^2 = (\Phi\theta - y)^T (\Phi\theta - y)$$

$$\therefore \nabla_{\theta} J(\theta) = \nabla_{\theta} [(\Phi\theta - y)^T (\Phi\theta - y)] = 0 \text{ to find minimizer.}$$

we know $\nabla(v^T v) = 2J_v^T v$ where $v = \Phi\theta - y$ & $J_v = \Phi$

$$\nabla_{\theta} J(\theta) = 2\Phi^T (\Phi\theta - y) = 0$$

$$\therefore 2\Phi^T (\Phi\hat{\theta} - y) = 0 \text{ for optimal } \hat{\theta}$$

$$\therefore \Phi^T \Phi \hat{\theta} - \Phi^T y = 0$$

$$\therefore \Phi^T \Phi \hat{\theta} = \Phi^T y$$

Given $\Phi^T \Phi$ is invertible coz, $\sum_{i=1}^N \phi(s^i, a^i) \phi^T(s^i, a^i)$ forms invertible matrix

$$\therefore \hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y$$

is given.
i.e. equivalent
to $\Phi^T \Phi$

this is a closed form

$$\hat{\theta} = \left(\sum_{i=1}^N \phi(s^i, a^i) \phi^T(s^i, a^i) \right)^{-1} \cdot \left(\sum_{i=1}^N \phi(s^i, a^i) y^i \right)$$

Answer

— x —

P3 To prove

$$\mathbb{E}_{S_{t+1} \sim P(\cdot | S_t, A_t)} \left[\sum_{t=0}^T R(S_t, A_t) \mid S_0 = s, A_0 = a \right] = Q^{\pi}(s, a) \quad \text{--- (A)}$$

LHS RHS

$A_t \sim \pi(\cdot | S_t)$
 $T \sim \text{Geo}(1-\gamma)$

Let the horizon length T follow geometric distribution & have parameter $(1-\gamma)$ defined over $\mathbb{N}_0$. Then pmf

$$\mathbb{P}[T = k] = (1-\gamma)\gamma^k, \quad k = 0, 1, 2, \dots \quad \text{--- (1)}$$

$$\mathbb{E} \left[\sum_{t=0}^T R(S_t, A_t) \right] = \sum_{k=0}^{\infty} \mathbb{P}[T = k] \cdot \mathbb{E} \left[\sum_{t=0}^k R(S_t, A_t) \mid T = k \right] \quad \text{--- (2)}$$

Since T is independent of MDP trajectory

$$\mathbb{E} \left[\sum_{t=0}^k R(s_t, A_t) \mid T=k \right] = \mathbb{E} \left[\sum_{t=0}^k R(s_t, A_t) \right]$$

$$\therefore \text{eqn } \textcircled{1} \text{ becomes } \mathbb{E} \left[\sum_{t=0}^T R(s_t, A_t) \right] = \mathbb{E} \left[\sum_{t=0}^k R(s_t, A_t) \right] \\ = \sum_{k=0}^{\infty} \mathbb{P}[T=k] \mathbb{E} \left[\sum_{t=0}^k R(s_t, A_t) \right] \quad \text{--- } \textcircled{3}$$

Substⁿ eqn $\textcircled{1}$ pmf $\textcircled{3}$ in $\textcircled{3}$

$$\mathbb{E} \left[\sum_{t=0}^T R(s_t, A_t) \right] = \sum_{k=0}^{\infty} (1-\gamma) \gamma^k \mathbb{E} \left[\sum_{t=0}^k R(s_t, A_t) \right] \quad \text{--- } \textcircled{4} \\ = \sum_{k=0}^{\infty} (1-\gamma) \gamma^k \sum_{t=0}^k \mathbb{E} [R(s_t, A_t)] \quad \text{--- } \textcircled{5}$$

when $k=0$, the RHS of eqn $\textcircled{5}$

$$(1-\gamma) \gamma^0 \sum_{t=0}^0 \mathbb{E} [R(s_t, A_t)] = (1-\gamma) \mathbb{E} [R(s_0, A_0)]$$

for $k=1$,

$$(1-\gamma) \gamma^1 \sum_{t=0}^1 \mathbb{E} [R(s_t, A_t)] = (1-\gamma) \gamma \left(\mathbb{E} [R(s_0, A_0)] + \mathbb{E} [R(s_1, A_1)] \right)$$

for $k=2$

$$(1-\gamma) \gamma^2 \sum_{t=0}^2 \mathbb{E} [R(s_t, A_t)] = (1-\gamma) \gamma^2 \left(\mathbb{E} [R(s_0, A_0)] + \mathbb{E} [R(s_1, A_1)] + \mathbb{E} [R(s_2, A_2)] \right)$$

$k=3$

$$(1-\gamma) \gamma^3 \sum_{t=0}^3 \mathbb{E} [R(s_t, A_t)] = (1-\gamma) \gamma^3 \left(\mathbb{E} [R(s_0, A_0)] + \mathbb{E} [R(s_1, A_1)] + \mathbb{E} [R(s_2, A_2)] + \mathbb{E} [R(s_3, A_3)] \right)$$

...

$\therefore$ LHS of eqn $\textcircled{5}$ substitute RHS with above expressions

$$\mathbb{E} \left[\sum_{t=0}^T R(s_t, A_t) \right] = (1-\gamma) \mathbb{E} [R(s_0, A_0)] \\ + (1-\gamma) \gamma \left(\mathbb{E} [R(s_0, A_0)] + \mathbb{E} [R(s_1, A_1)] \right) \\ + (1-\gamma) \gamma^2 \left(\mathbb{E} [R(s_0, A_0)] + \mathbb{E} [R(s_1, A_1)] + \mathbb{E} [R(s_2, A_2)] \right) \\ + \dots \quad \text{--- } \textcircled{6}$$

grouping terms,

(7)

total coeff: in front of $\mathbb{E}[R(s_0, A_0)]$ is

$$\text{for } k = \begin{matrix} 0 & 1 & 2 & 3 & \dots \end{matrix} \quad (1-\gamma) + (1-\gamma)\gamma + (1-\gamma)\gamma^2 + (1-\gamma)\gamma^3 + \dots$$

$$\therefore (1-\gamma) (1 + \gamma + \gamma^2 + \gamma^3 + \dots) = (1-\gamma) \frac{1}{1-\gamma} = 1.$$

↑
geo series.

$\therefore$ coeff of $\mathbb{E}[R(s_0, A_0)]$ is 1.

total coeff for $\mathbb{E}[R(s_1, A_1)]$

$$(1-\gamma)\gamma + (1-\gamma)\gamma^2 + \dots$$

$$\therefore (1-\gamma)\gamma (1 + \gamma + \gamma^2 + \gamma^3 + \dots) = (1-\gamma) \cdot \gamma \frac{1}{1-\gamma} = \gamma$$

↓
geo series

$\therefore$ coeff of $\mathbb{E}[R(s_1, A_1)]$ is γ

total coeff for $\mathbb{E}[R(s_2, A_2)]$

$$(1-\gamma)\gamma^2 + (1-\gamma)\gamma^3 + (1-\gamma)\gamma^4 + \dots$$

$$(1-\gamma)\gamma^2 (1 + \gamma + \gamma^2 + \dots) = (1-\gamma) \cdot \gamma^2 \frac{1}{1-\gamma} = \gamma^2$$

↓
geo series

$\therefore$ coeff of $\mathbb{E}[R(s_2, A_2)]$ is γ^2

so coeff of $\mathbb{E}[R(s_t, A_t)]$

$$(1-\gamma)\gamma^t + (1-\gamma)\gamma^{t+1} + (1-\gamma)\gamma^{t+2} + \dots$$

$$= (1-\gamma)\gamma^t \frac{1}{1-\gamma} = \gamma^t$$

$\therefore$ eqn (6) becomes

$$\mathbb{E}\left[\sum_{t=0}^T R(s_t, A_t)\right] = 1 \cdot \mathbb{E}[R(s_0, A_0)] + \gamma \mathbb{E}[R(s_1, A_1)] + \gamma^2 \mathbb{E}[R(s_2, A_2)] + \dots$$

$$\therefore \mathbb{E} \left[\sum_{t=0}^T R(s_t, A_t) \right] = \sum_{t=0}^{\infty} \gamma^t \mathbb{E} [R(s_t, A_t)] \quad \text{--- (7)}$$

(8)

we start with LHS of (7)

$$\therefore \mathbb{E} \left[\sum_{t=0}^T R(s_t, A_t) \mid S_0 = s, A_0 = a \right] = \sum_{t=0}^{\infty} \gamma^t \mathbb{E} [R(s_t, A_t) \mid S_0 = s, A_0 = a]$$

[Conditioning to $S_0 = s$ & $A_0 = a$]

LHS

$$= \mathbb{E}_{\substack{S_{t+1} \sim P(\cdot | S_t, A_t) \\ A_t \sim \pi(\cdot | S_t) \\ T \sim \text{Geo}(1-\gamma)}} \left[\sum_{t=0}^{\infty} \gamma^t \mathbb{E} [R(s_t, A_t) \mid S_0 = s, A_0 = a] \right]$$

by linearity of expectation

$$= \mathbb{E} [Q^{\pi}(s, a)] \text{ by definition.}$$

hence

$$\mathbb{E}_{\substack{S_{t+1} \sim P(\cdot | S_t, A_t) \\ A_t \sim \pi(\cdot | S_t) \\ T \sim \text{Geo}(1-\gamma)}} \left[\sum_{t=0}^T R(s_t, A_t) \mid S_0 = s, A_0 = a \right] = \mathbb{E}_{\text{LHS}} [Q^{\pi}(s, a)] = Q^{\pi}(s, a) \text{ by definition.}$$

Q.E.D. "phew!"

P 4.1

Given the class of state-action value function Q

$$Q = \{ \theta Q_{\theta} : Q_{\theta}(s, a) = \theta_1 s^2 + \theta_2 a^2 + \theta_3 s a + \theta_4, \theta \in \mathbb{R}^4 \}$$

data set

i	s^i	a^i	y^i
1	1	0	1
2	-2	1	0
3	1	-1	3

ERM square loss objective is,

$$\arg \min_{\theta \in \mathbb{R}^4} L(\theta_1, \theta_2, \theta_3, \theta_4) = \min_{\theta \in \mathbb{R}^4} \sum_{i=1}^3 (Q_{\theta}(s^i, a^i) - y^i)^2$$

∴ the ERM objective is

$$\arg \min_{\theta \in \mathbb{R}^4} \sum_{i=1}^3 (\theta_1 s_i^2 + \theta_2 a_i^2 + \theta_3 s_i a_i + \theta_4 - y_i)^2 \quad \text{--- (1)}$$

Now we plug in 3 data points

1. $\rightarrow (s^1, a^1, y^1) = (1, 0, 1)$

$$\theta_1 (1)^2 + \theta_2 (0)^2 + \theta_3 (1) \cdot (0) + \theta_4 - 1 = \theta_1 + \theta_4 - 1$$

2. $\rightarrow (s^2, a^2, y^2) = (-2, 1, 0)$

$$\theta_1 (-2)^2 + \theta_2 (1)^2 + \theta_3 (-2) \cdot (1) + \theta_4 - 0 = 4\theta_1 + \theta_2 - 2\theta_3 + \theta_4$$

3. $(s^3, a^3, y^3) = (1, -1, 3)$

$$\theta_1 (1)^2 + \theta_2 (-1)^2 + \theta_3 (1) (-1) + \theta_4 - 3 = \theta_1 + \theta_2 - \theta_3 + \theta_4 - 3$$

∴ plugging in eqn (1)

$$\arg \min_{\theta \in \mathbb{R}^4} \left[(\theta_1 + \theta_4 - 1)^2 + (4\theta_1 + \theta_2 - 2\theta_3 + \theta_4)^2 + (\theta_1 + \theta_2 - \theta_3 + \theta_4 - 3)^2 \right]$$

Answer

Q. 6

Answer 1.1:

```
In [1]: import numpy as np
from scipy.linalg import solve
from IPython.display import display, Markdown

# Define MDP
states = [1, 2, 3]
actions = ['g', 'h']
gamma = 0.95
initial_state = 1

# Transition probabilities  $P(s'|s,a)$  -> Format:  $P[s][a][s']$ 
P = {
    1: {'g': {1: 0.1, 2: 0.8, 3: 0.1}, 'h': {1: 0.8, 2: 0.1, 3: 0.1}},
    2: {'g': {1: 0.1, 2: 0.1, 3: 0.8}, 'h': {1: 0.1, 2: 0.8, 3: 0.1}},
    3: {'g': {1: 0.8, 2: 0.1, 3: 0.1}, 'h': {1: 0.1, 2: 0.1, 3: 0.8}}
}

# Reward function  $R(s, a)$ 
def reward(s, a):
    return 1.0 if (s == 3 and a == 'h') else 0.0

# Target policy  $\pi^t$ 
pi_t = {
    1: {'g': 0.9, 'h': 0.1},
    2: {'g': 0.9, 'h': 0.1},
    3: {'g': 0.1, 'h': 0.9}
}

# Behavior policy  $\pi^b$ 
pi_b = {
    1: {'g': 0.85, 'h': 0.15},
    2: {'g': 0.88, 'h': 0.12},
    3: {'g': 0.1, 'h': 0.9}
}

def compute_value_function(policy):
    """
    Compute the value function  $V^{\pi}$  using the Bellman equation.
    """
    n_states = len(states)

    # Expected transition matrix  $P_{\pi}$  and expected reward vector  $R_{\pi}$ 
    P_pi = np.zeros((n_states, n_states))
    R_pi = np.zeros(n_states)

    for i, s in enumerate(states):
        for a in actions:
            pi_sa = policy[s][a]
            R_pi[i] = R_pi[i] + pi_sa * reward(s, a)

            for j, s_prime in enumerate(states):
                P_pi[i, j] = P_pi[i, j] + pi_sa * P[s][a][s_prime]
```

```

        # print(P_pi)

        # Solve  $(I - \gamma P_{\pi})V = R_{\pi}$ 
        I = np.eye(n_states)
        A = I - gamma * P_pi
        V = solve(A, R_pi)

    return V

V_target_exact = compute_value_function(pi_t)
V_behavior_exact = compute_value_function(pi_b)

# Fancy display to impress everyone
# Compute exact values using model
display(Markdown("***Exact Values (Model-Based)***"))

display(Markdown("***Target Policy  $\pi^t$ ***"))
for i, s in enumerate(states):
    display(Markdown(f" $V^{\pi^t}(\{s\}) = \{V\_target\_exact[i]:.4f\}$ "))
    # print(f"  $V^{\pi_t}(\{s\}) = \{V\_target\_exact[i]:.4f\}$ ")

display(Markdown("***Target Policy  $\pi^b$ ***"))
for i, s in enumerate(states):
    display(Markdown(f" $V^{\pi^b}(\{s\}) = \{V\_behavior\_exact[i]:.4f\}$ "))
    # print(f"  $V^{\pi_b}(\{s\}) = \{V\_behavior\_exact[i]:.4f\}$ ")

```

Exact Values (Model-Based)

Target Policy π^t

$$V^{\pi^t}(1) = 10.3676$$

$$V^{\pi^t}(2) = 10.9210$$

$$V^{\pi^t}(3) = 11.7842$$

Target Policy π^b

$$V^{\pi^b}(1) = 10.2405$$

$$V^{\pi^b}(2) = 10.8086$$

$$V^{\pi^b}(3) = 11.6824$$

Answer 1.2:

```

In [2]: epsilon = 0.1

# Compute R_max from reward(s, a) function earlier
R_max = max(reward(s, a) for s in states for a in actions)

def compute_effective_horizon(gamma, epsilon, R_max):
    """
    Compute effective horizon T such that the truncation error < epsilon
    We use the bound:
        error ≤ R_max * gamma^T / (1 - gamma)
    and solve for T:
    """

```

```

        T ≥ log(R_max / (epsilon * (1 - gamma))) / log(1 / gamma)
    """
    T = int(
        np.ceil(np.log(R_max / (epsilon * (1 - gamma))) / np.log(1 / gamma))
    )
    return T
# Make fancy display to impress everyone
T_effective = compute_effective_horizon(gamma, epsilon, R_max)
display(Markdown(f" Effective Horizon T = {T_effective}"))

```

Effective Horizon T = 104

Answer 1.3:

```

In [3]: def generate_trajectory(start_state, policy, horizon):
    """
    Generate a trajectory of fixed length following the given policy.
    """
    trajectory = []
    state = start_state

    for t in range(horizon):
        # Sample action from policy
        action = np.random.choice(actions, p=[policy[state]['g'], policy[state]['h']])

        # Get reward
        r = reward(state, action)

        # Sample next state
        next_state = np.random.choice(states, p=[P[state][action][s] for s in states])

        trajectory.append((state, action, r))
        state = next_state

    return trajectory

def off_policy_mc_finite(start_state, behavior_policy, target_policy,
                        num_trajectories, horizon):
    """
    Off-policy Monte Carlo evaluation with finite trajectories.
    Uses weighted importance sampling.
    """
    returns = []
    weights = []

    for i in range(num_trajectories):
        trajectory = generate_trajectory(start_state, behavior_policy, horizon)

        # Compute return G for this trajectory
        G = 0
        for t in range(len(trajectory) - 1, -1, -1):
            _, _, r = trajectory[t]
            G = gamma * G + r

        # Compute importance sampling ratio W

```

```

W = 1.0
for state_t, action_t, _ in trajectory:
    pi_target = target_policy[state_t][action_t]
    pi_behavior = behavior_policy[state_t][action_t]
    W *= pi_target / pi_behavior

returns.append(G)
weights.append(W)

# Weighted importance sampling estimate
returns = np.array(returns)
weights = np.array(weights)

if np.sum(weights) > 0:
    V_hat = np.sum(weights * returns) / np.sum(weights)
else:
    V_hat = 0.0

return V_hat, returns, weights

num_trajectories = 50 # Provided
np.random.seed(1)

V_hat_target, returns, weights = off_policy_mc_finite(
    start_state=1,
    behavior_policy=pi_b,
    target_policy=pi_t,
    num_trajectories=num_trajectories,
    horizon=T_effective
)

# Make fancy display to impress everyone
display(Markdown(
    f"### Evaluating target policy  $\hat{V}^{\pi^t}$  using importance sampling"
))
display(Markdown(
    f"Estimated value function:  $\hat{V}^{\pi^t}(1) = \{V\_hat\_target:.4f\}$ "
))

```

Evaluating target policy $\hat{V}^{\pi^t}$ using importance sampling

Estimated value function: $\hat{V}^{\pi^t}(1) = 10.4068$

Answer 1.4:

```

In [4]: # True infinite-horizon value from Part 1.1
V_true = V_target_exact[0]

# Estimated value from Part 1.3
V_est = V_hat_target

# Error
abs_error = abs(V_est - V_true)

```

```

# Make fancy display to impress everyone
display(Markdown(
    f"""
    **True value:**   $V^{\pi^t}(1) = \{V\_true:.4f\},$ 
    **Estimated value:**   $\hat{V}^{\pi^t}(1) = \{V\_est:.4f\},$ 
    **and Absolute error:**   $\{abs\_error:.4f\}$ 
    """
))

```

True value: $V^{\pi^t}(1) = 10.3676$, **Estimated value:** $\hat{V}^{\pi^t}(1) = 10.4068$, **and Absolute error:** 0.0392

Answer 4.2

```

In [5]: # State and action spaces
state_space = np.arange(-3, 4)
action_space = np.arange(-1, 2)

# Given Q-hat
def qhat0(s, a):
    return 2*s**2 + a**2 - s*a + 0.5

# Random tie-breaking greedy action
def greedy_action_with_ties(Q_state):
    values = np.array(list(Q_state.values()))
    actions = list(Q_state.keys())
    max_val = values.max()
    best_idxs = np.where(values == max_val)[0]
    chosen_idx = np.random.choice(best_idxs)
    return actions[chosen_idx]

# Build uniform pi0
pi0 = {s: {a: 1/3 for a in action_space} for s in state_space}

# Build greedy p-bar(s)
pbar = {}
for s in state_space:
    Qs = {a: qhat0(s, a) for a in action_space}
    a_star = greedy_action_with_ties(Qs)
    pbar[s] = {a: (1.0 if a == a_star else 0.0) for a in action_space}

# Form pi1 = (1 - alpha)pi0 + alpha p-bar
alpha = 0.25
pi1 = {}

for s in state_space:
    pi1[s] = {}
    for a in action_space:
        pi1[s][a] = (1 - alpha)*pi0[s][a] + alpha*pbar[s][a]

# Pretty display
def show_pi1_state(s):
    dist = pi1[s]
    display(Markdown(
        rf"""

```

```

###  $\pi_1(s = \{s\})$ 

| action \ (a\ |  $\pi_1(a \mid s=\{s\})$  |
|-----:-----:|
| -1 | {dist[-1]:.2f} |
| 0 | {dist[0]:.2f} |
| 1 | {dist[1]:.2f} |
"""

))

show_pi1_state(-1)
show_pi1_state(0)
show_pi1_state(2)

```

$$\pi_1(s = -1)$$

action (a)	$\pi_1(a \mid s=-1)$
-1	0.25
0	0.25
1	0.50

$$\pi_1(s = 0)$$

action (a)	$\pi_1(a \mid s=0)$
-1	0.25
0	0.25
1	0.50

$$\pi_1(s = 2)$$

action (a)	$\pi_1(a \mid s=2)$
-1	0.50
0	0.25
1	0.25

Answer 5.1:

```
In [6]: gamma = 0.9
epsilon = 0.1

states = ["b", "c", "d", "e"]
actions = ["x", "y"]

Q = {
    ("b", "x"): -1.5,  ("b", "y"): -2.5,
```

```

        ("c", "x"): -0.5, ("c", "y"): -1.0,
        ("d", "x"): 0.0, ("d", "y"): -0.25,
        ("e", "x"): -0.5, ("e", "y"): 0.75,
    }
}
def argmax(d):
    best_key = None
    best_val = -np.inf
    for k, v in d.items():
        if v > best_val:
            best_val = v
            best_key = k
    return best_key

def get_epsilon_greedy_policy(Q, states, actions, epsilon):
    policy = {}

    for s in states:
        q_values = {a: Q[(s, a)] for a in actions}
        max_action = argmax(q_values)

        print(f"\nState {s}:")
        print(f"  Q({s}, x) = {Q[(s, 'x')]:.4f}, Q({s}, y) = {Q[(s, 'y')]:.4f}")
        print(f"  Best action: {max_action}")

        #  $\epsilon$ -greedy assignment
        for a in actions:
            if a == max_action:
                policy[(s, a)] = 1 - epsilon + epsilon / len(actions)
            else:
                policy[(s, a)] = epsilon / len(actions)

        print(f"pi(x|{s}) = {policy[(s, 'x')]:.4f}, pi(y|{s}) = {policy[(s, 'y')]:.4f}")

    return policy

policy = get_epsilon_greedy_policy(Q, states, actions, epsilon)
formatted_policy = {k: round(v, 2) for k, v in policy.items()}
print("")
# Make fancy display to impress everyone
table = "| $State$ | $Action$ | $\pi(a|s)$ |\n"
table += "|-----|-----|-----|\n"

for s in states:
    for a in actions:
        p = formatted_policy[(s, a)]
        table += f"| {s} | {a} | {p:.2f} |\n"

display(Markdown(table))

```

$Q(b, x) = -1.5000$, $Q(b, y) = -2.5000$
 Best action: x
 $\pi(x|b) = 0.9500$, $\pi(y|b) = 0.0500$

$Q(c, x) = -0.5000$, $Q(c, y) = -1.0000$
 Best action: x
 $pi(x|c) = 0.9500$, $pi(y|c) = 0.0500$

Q(d, x) = 0.0000, Q(d, y) = -0.2500
Best action: x
pi(x|d) = 0.9500, pi(y|d) = 0.0500

$Q(e, x) = -0.5000$, $Q(e, y) = 0.7500$
 Best action: y
 $\pi(x|e) = 0.0500$, $\pi(y|e) = 0.9500$

Answer 5.2:

```
In [10]: # Given sub-trajectory
trajectory = [
    ('c', 'x', 1.0, 'e'),
    ('e', 'x', -2.0, 'b'),
    ('b', 'y', -0.5, 'b')
]

def create_dataset(Q, trajectory, gamma):
    """Creates (input, target) pairs for supervised learning."""
    dataset = []

    for (s_t, a_t, r_t, s_next) in trajectory:

        # Input pair
        data_input = (s_t, a_t)

        # Max over next actions:  $\max_{a'} Q(s_{next}, a')$ 
        max_q_next = max([Q[(s_next, a)] for a in actions])

        # TD target
        target = r_t + gamma * max_q_next

        dataset.append({
            'input': data_input,
            'target': target
        })

    return dataset
```

```

dataset = create_dataset(Q, trajectory, gamma)

# Make fancy display to impress everyone
display(Markdown(f"""**Dataset**` {dataset}` """))

table = "| State | Action | Target |\n"
table += "|-----|-----|-----|\n"

for d in dataset:
    s, a = d["input"]
    y = d["target"]
    table += f"|{s}|{a}|{y:.4f}|\n"

display(Markdown(table))

```

Dataset [{'input': ('c', 'x'), 'target': 1.675}, {'input': ('e', 'x'), 'target': -3.35}, {'input': ('b', 'y'), 'target': -1.85}]

State	Action	Target
c	x	1.6750
e	x	-3.3500
b	y	-1.8500

Answer 5.4:

```

In [11]: # Feature definitions from the problem
def phi1(s, a):
    # phi1(s,a) = -2.1[s=b] - 1.1[s=c] - 0.5.1[s=d]
    return -2.0 * (s == "b") - 1.0 * (s == "c") - 0.5 * (s == "d")

def phi2(s, a):
    # phi2(s,a) = 1.1[a=x] - 1.1[a=y]
    return 1.0 * (a == "x") - 1.0 * (a == "y")

def phi(s, a):
    return np.array([phi1(s, a), phi2(s, a)], dtype=float)

# Build matrix phi and target vector y from the dataset of Part 2
Phi = []
y_vec = []

for d in dataset:
    s, a = d["input"]
    target = d["target"]
    Phi.append(phi(s, a))
    y_vec.append(target)

Phi = np.vstack(Phi)
y_vec = np.array(y_vec)

# Solve least squares: min_theta ||phi theta - y||^2
theta_hat, residuals, rank, svals = np.linalg.lstsq(Phi, y_vec, rcond=None)

```

```

theta1_hat, theta2_hat = theta_hat

# Compute predictions and empirical mean square loss
y_pred = Phi @ theta_hat
ms = (1 / len(y_vec)) * np.sum((y_pred - y_vec) ** 2)
# Compute Sum of Squared Loss (without dividing by N as TA mentioned)
ss = np.sum((y_pred - y_vec)**2)

# Make fancy display to impress everyone
display(Markdown(
    rf"""
    ### Optimal Parameters

    $\theta_1^* = \{{theta1\_hat:.6f}\},$
    $\theta_2^* = \{{theta2\_hat:.6f}\},$
    **Mean Squared (MS):** $L_{\{\text{MS}\}}(\theta^*) = \{{ms:.6f}\}$
    **and Sum of Squared (SS):** $L_{\{\text{SS}\}}(\theta^*) = \{{ss:.6f}\}$
    """)
))

table = rf"""### Predictions vs Targets

| State | Action | Target $y$ | Prediction $Q_{\theta}(s,a)$ |
|-----|-----|-----|-----|
"""

for i, d in enumerate(dataset):
    s, a = d["input"]
    y_true = y_vec[i]
    y_hat = y_pred[i]
    table += f"| {s} | {a} | {y_true:.4f} | {y_hat:.4f} |\n"

display(Markdown(table))

```

Optimal Parameters

$\theta_1^* = 0.421429$, $\theta_2^* = -0.082143$, **Mean Squared (MS):** $L_{\text{MS}}(\theta^*) = 5.537202$ and **Sum of Squared (SS):** $L_{\text{SS}}(\theta^*) = 16.611607$

Predictions vs Targets

State	Action	Target y	Prediction $Q_{\theta}(s, a)$
c	x	1.6750	-0.5036
e	x	-3.3500	-0.0821
b	y	-1.8500	-0.7607